The new line in initialize is where the magic happens. To understand it, you need to know a couple of things.

First, if you pass a block to Array#grep, the block is evaluated for each element that matches the regular expression. Second, the string matching the parenthesized part of the regular expression is stored in the global variable $1. So, if data_source has methods named get_cpu_info and get_mouse_info, this code ultimately calls Computer.define_component twice, with the strings "cpu" and "mouse". Note that define_method works equally well with a string or a symbol.

The duplicated code is finally gone for good. As a bonus, you don't even have to write or maintain the list of components. If someone adds a new component to DS, the Computer class will support it automatically. Isn't that wonderful?

Let's Try That Again

Your refactoring was a resounding success, but Bill is not willing to stop here. "We said that we were going to try *two* different solutions to this problem, remember? We've only found one, involving *Dynamic Dispatch* *(49)* and *Dynamic Methods* *(51)*. It did serve us well—but to be fair, we need to give the other solution a chance."

For this second solution, you need to know about some strange methods that are not really methods and a very special method named method_missing.

method_missing

Where you listen to spooky stories about Ghost Methods and dynamic proxies and you try a second way to remove duplicated code.

With Ruby, there's no compiler to enforce method calls. This means you can call a method that doesn't exist. For example:

methods/method_missing.rb

```
class Lawyer; end
nick = Lawyer.new
nick.talk_simple
```

```
NoMethodError: undefined method `talk_simple' for #<Lawyer:0x007f801aa81938>
```

Do you remember how method lookup works? When you call talk_simple, Ruby goes into nick's class and browses its instance methods. If it can't find talk_simple there, it searches up the ancestors chain into Object and eventually into BasicObject.

Because Ruby can't find talk_simple anywhere, it admits defeat by calling a method named method_missing on nick, the original receiver. Ruby knows that